Multi-Agent Research Presentation Synthesis with Critic-Guided Revision

Sheng Rao
University of South Florida
srao2@usf.edu

Vien Nguyen
University of South Florida
viennguyen@usf.edu

Abstract

Research papers are often written for careful reading, while presentations are built for quick, guided consumption. Converting one into the other requires more than shortening paragraphs into bullet points: a system must decide what argument to tell, what evidence supports that argument, and how to revise slides when the first draft is weak. We present the Multi-Agent Research Presentation Synthesizer, a LangGraph pipeline that converts one or more research paper PDFs into a PowerPoint deck through document processing, presentation planning, parallel slide drafting / critic review loop, targeted rewriting, and Pandoc export. The system can parse multiple source materials, contextualize text chunks and artifacts with embeddings for RAG. The system also supports query-driven goals, including explanation, comparison, and criticism of a paper's methodology. Evaluation is planned using DECKBench.

1 Introduction

Between reading/understanding a research paper and presenting a research paper for the understanding of an audience, the first requires attention to dense, recursive details that a reader may revisit, while the second requires a narrative flow of presentation that assists in carrying the presenter's chosen thesis to their target audience. A presentation has to plan a path through the material, make the audience care about that path, and decide which details should become visible evidence instead of background knowledge. This difference matters especially for technical papers, where a weak slide deck can flatten a careful contribution into a list of disconnected claims.

Large language models make this problem look easier than it is: A model can summarize general portions of the paper, extract contribution bullets, and even generate simple title-and-content slides from a PDF. However, such outputs often have hallucinations or consistency mismatches between slide and paper, issues with narrative flow, or issues with incorrect or incompatible slide layouts. If a generated slide says that a method improves robustness, such information should be confirmed to be grounded in the original paper(s). If a slide deck repeats the same idea two times while skipping the main limitation or drawback, a critic should catch the narrative failure. If a figure or equation is central to the paper, the export layer should preserve it in a form that a presenter can actually use. These requirements make the task an orchestration problem instead of a single-shot generation task.

Our project, the Multi-Agent Research Presentation Synthesizer, explores this orchestration view. Given one or more research PDFs and a user query, the system produces a PowerPoint presentation through a multi-stage LangGraph workflow. The query can define the audience and purpose of the deck; for example, the same input paper can be explained to undergraduate students, compared against another paper, or criticized for a senior research audience. This query-driven framing is important because presentation quality is not universal: a clear teaching deck, a skeptical lab-meeting deck, and a comparison deck have different narrative obligations.

The system makes four main design choices. First, it separates work into specialized roles: a Planner creates the deck thesis and slide blueprints, Slide Writers draft or rewrite slide groups, Critics check grounding and narrative coherence, and a Supervisor decides when and how to continue/end the critic-rewrite cycle. Second, it supports multiple input PDFs so the final deck can synthesize a small body of work instead of only summarizing one document. Third, it keeps a persistent SQLite workspace containing parsed documents, text chunks, embeddings, images, tables, equations, proto-slides, and review events to assist in contextualizing the ingested documents and subsequent

Retrieval Augmented Generation. Finally, it exports to native `.pptx` through Pandoc, preserving speaker notes, Markdown structure, math, and media where possible.

2 Related Work

Recent systems for automatic presentation generation have moved beyond simple text-to-slides prompting. The closest work uses agentic workflows, reflection, reference decks, or rendering feedback to create research presentations. Our system is in the same line of work, but puts together a different set of tradeoffs: review state, multi-document input, query-driven presentation intent, and editable PowerPoint export.

Yang et al. (2025) present Auto-Slides, an interactive multi-agent system for creating and customizing research presentations. Auto-Slides is an important reference point because it treats academic slide generation as a multi-agent process rather than a single summarization call. Its output target is LaTeX Beamer, which is natural for many academic users and gives strong typesetting support. That choice creates friction when users need editable PowerPoint files, which is common in office or academic settings. The public documentation also indicates practical dependencies such as `marker-pdf`, which can require substantial memory, and verification or repair steps that may be slow enough for users to bypass. More importantly for our project, Auto-Slides processes only one pdf and focuses on incorporating user input near the end of the process (allowing the user to order specific changes to the slide content, flow, or layout), while our system supports combining several PDFs into one deck and changing the presentation goal through an explicit query such as "criticize this paper's methodology."

PPTAgent takes a different direction. Zheng et al. (2025) frame presentation generation as an edit-based process inspired by how humans adapt existing slide designs. The system analyzes reference presentations, clusters slides by function, extracts schemas, and then generates new slides through editing operations. This is useful because visual design is not an afterthought in real presentation work. PPTAgent also introduces PPTEval, an evaluation framework for presentation content, design, and coherence. Its limitation, from our perspective, is that the method depends strongly on reference presentations for both style and structure. This can help visual quality when good references are available, but it also makes the system less self-contained. The paper and related descriptions also note that PowerPoint's XML representation and nested visual elements are difficult to manipulate reliably, so layout inconsistency and overlapping elements can remain even in a reflective framework. In contrast, our system accepts more modest visual control in exchange for a clearer source-grounded synthesis pipeline.

Zheng et al. (2026) propose DeepPresenter, an agentic framework based on environment-grounded reflection. Instead of reflecting only over hidden reasoning traces or textual self-critiques, DeepPresenter renders intermediate slide artifacts and uses perceptual observations of those artifacts during revision. This is a strong idea for slide generation because many presentation problems, such as crowding, poor alignment, or missing visual hierarchy, are visible only after rendering. The tradeoff is infrastructure complexity: the system needs a reliable render environment at inference time and includes a fine-tuned 9B model that requires local inference. However, the use of a single model to both produce the slides and evaluate those slides introduces concerns of circular feedback, as models often have issues identifying flaws in their own output. DeepPresenter also uses AI image generation to add visual flair to its presentation slides, which adds additional concerns of hallucinations in those images (especially in text within those images) alongside the questionable relevance of images generated for the slides they are placed in. DeepPresenter is therefore valuable evidence that visual feedback matters, but it has accompanying issues with its image generation and circular feedback. DeepPresenter also outputs slides in HTML, not Powerpoint.

DECKBench is also relevant, mainly as an evaluation benchmark. Jang et al. (2026) introduce a benchmark for academic paper-to-slide generation and multi-turn slide editing. It evaluates the full workflow from generating a deck from a paper to refining it through natural-language editing instructions, with both reference-free and reference-based metrics. This makes DECKBench a natural future evaluation target for our system, especially because our revision loop already resembles the benchmark's interest in multi-turn editing. At the time of this report, DECKBench results have not yet been run for our project, so we cite it as the intended evaluation framework instead of completed

evidence.

These systems make different bets. Auto-Slides emphasizes interactive academic slide creation, PPTAgent emphasizes reference-guided PowerPoint design, and DeepPresenter emphasizes rendered-environment feedback. Our project keeps the visual side simpler and instead combines explicit grounding, review-state tracking across cycles, multiple-document synthesis, and a query-controlled path to PowerPoint.

3 System Design

3.1 Overview

The system is organized as a pipeline from evidence and artifact extraction to deck export. A run begins with one or more PDF files and an optional user query. The PDFs are processed into structured document records, chunks, artifacts, and those elements contextualized into embeddings. The Planner then reads the available evidence and creates a presentation plan. Slide Writers draft groups of slides in parallel. Critics review the resulting deck for source grounding and narrative coherence. A Supervisor uses these reviews to decide whether to accept the deck, request targeted rewrites, or return to planning. Once accepted, the export layer converts the latest proto-slides into a PowerPoint file.

The user query is part of the presentation specification, not prompt decoration. A query such as *Explain this paper to an audience of laypeople* should produce different slide blueprints from *Make a presentation pointing out this paper's flaws to senior researchers*. The Planner therefore treats the query as a constraint on audience, tone, and narrative purpose. This allows the same evidence store to support explanation, comparison, criticism, or other forms of audience-specific communication.

3.2 Document Processing and Evidence Store

Document processing happens before the LangGraph workflow begins. The active OCR and parsing backend is LlamaParse, which is used because research PDFs often contain equations, tables, captions, and layout-sensitive information that plain text extraction handles poorly. The processor follows a strategy pattern so that OCR backends can be swapped, but the current supported runtime path uses LlamaParse. Older local backends remain in the codebase for reference, but they are not the recommended path because they were slow or inaccurate in practice. We ultimately decided on LlamaParse because of its freedom from relying on local inference, its generally high accuracy in parsing figures, tables, and equations, and its price-competitive offering alongside its generous credit bonus on sign-up.

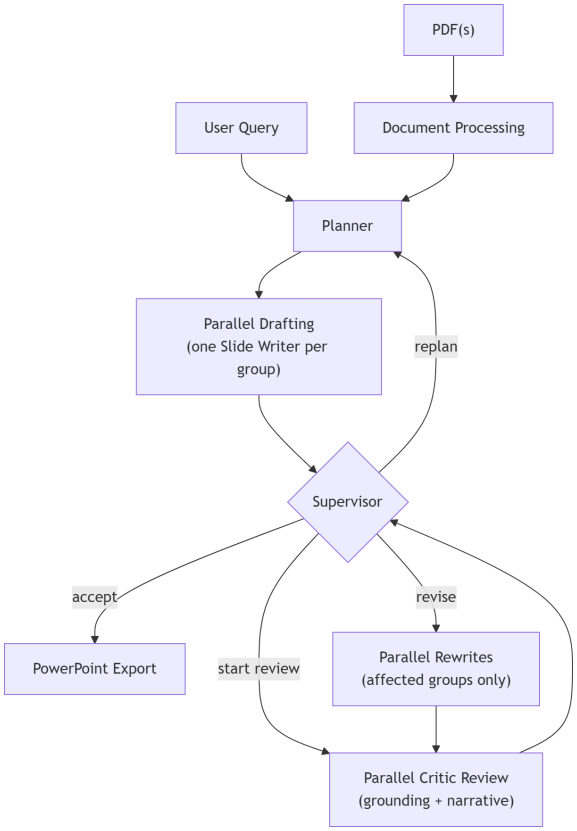

Figure 1: Full system pipeline from document ingestion to PowerPoint output

The processing stage first computes a SHA-256 hash of each PDF. If the same document has already been parsed, the system can load the cached document from SQLite instead of spending time and API calls on extraction again. On a cache miss, the backend extracts Markdown text, document metadata, images, tables, equations, page information, and layout artifacts. The resulting Markdown can be split semantically into smaller chunks, which is the default, or stored as a single chunk when chunking is disabled. Semantic chunking is useful because planning and retrieval both need evidence units that are preferably significantly smaller than sending the whole paper in order not to balloon the size of the prompt sent to the agent LLMs.

The system then contextualizes the evidence at two levels. A local contextualizer runs LLM calls over chunks and artifacts, producing

contextualized_text fields that improve embedding and full-text search. A separate document contextualizer produces a document-level structured summary such as section outline and paper metadata. These two contextualization passes serve different purposes: the first makes individual evidence items easier to retrieve, while the second gives planning a compact view of the paper's structure.

All evidence is persisted in a single SQLite database, data/research.db. The database stores document rows, text chunks, vector embeddings through sqlite-vec, images, tables, equations, full-text search indexes, retrieval logs, proto-slides, and review events. This design is especially important for multiple PDFs. Each source document remains a first-class object with its own chunks and artifacts, while the planning and drafting stages can reason over the combined evidence base. As a result, a deck can compare two papers or synthesize several sources without pretending that they came from one monolithic input.

3.3 Graph Architecture

The LangGraph workflow uses four agent roles and one deterministic coordination layer. The Planner is the presentation architect. It loads source chunks, detects section boundaries using Markdown headings, builds a human-readable outline with stable labels, and asks the LLM for a structured plan. The plan includes a title, subtitle, thesis, target audience, narrative arc, per-slide blueprints, and slide groupings (a way to split a slide deck into separate narratively-delineated elements). The model does not pass raw (numeric) chunk identifiers directly, instead, code resolves validated section labels back to concrete chunks after the LLM call. This keeps the prompt easier to reason about while preserving source linkage.

Slide Writers receive the blueprints and source context for an assigned slide group (with no overlap). In initial-write mode, a writer drafts slides from scratch according to the blueprint. In rewrite mode, the writer receives existing proto-slides plus explicit rewrite instructions produced from critic findings. Both modes write structured slide records back to the database. This structure includes slide titles, bullet points, optional sub-bullets, layout hints, speaker notes, and source references. Errors are handled without crashing the whole graph; if an initial group produces no slides, the deterministic executor can retry it before review begins.

Critics evaluate the current deck from two angles. Per-group Critics check grounding consistency: whether slide claims are supported by the relevant source material. A deck-level Critic checks narrative coherence: whether the presentation follows the plan, fits the user query, and creates a coherent arc for the intended audience. Each critic result includes an overview, a flag for whether action is required, and a typed issue list. Issues include severity, location, description, rewrite instruction, and affected slide numbers. The system also assigns each issue a fingerprint based on its scope, type, and location.

The Supervisor is the routing decision-maker. It runs after each fan-in point, when a batch of parallel workers has completed. If no critic results exist yet, or if rewrites have just completed, deterministic pre-LLM logic dispatches a new critic cycle. When fresh critic results exist, the Supervisor calls an LLM with critic summaries, severity counts, and recurring issue fingerprints. The model can choose accept, revise, or replan. Guard logic then corrects impossible or unhelpful decisions: for example, a request to revise with no actionable issues becomes acceptance, while repeated major problems at the cycle cap can force replanning when budget remains.

The graph is deliberately batch-oriented. Parallel slide writers and parallel critics do not independently mutate the route of the run. They return to a coordination checkpoint, and only then does the Supervisor decide the next action. This design avoids race conditions and makes the workflow easier to audit. It also matches careful group work: many people can draft or review at the same time, but someone still has to gather the results before deciding what to do next.

3.4 Review State and Revision Tracking

The system keeps agents stateless per call, but keeps the run stateful. This matters because an LLM call in cycle 3 does not automatically know what happened in cycles 1 and 2. If the system relied only on transient prompts, recurring problems could look new every time. Review events and compact summaries are therefore stored outside the model context in the storage database.

The proto_slides table stores the latest active version of each generated slide. When a rewrite happens, prior content and source references are preserved as previous revision snapshots. The best_proto_slides table stores the best-seen

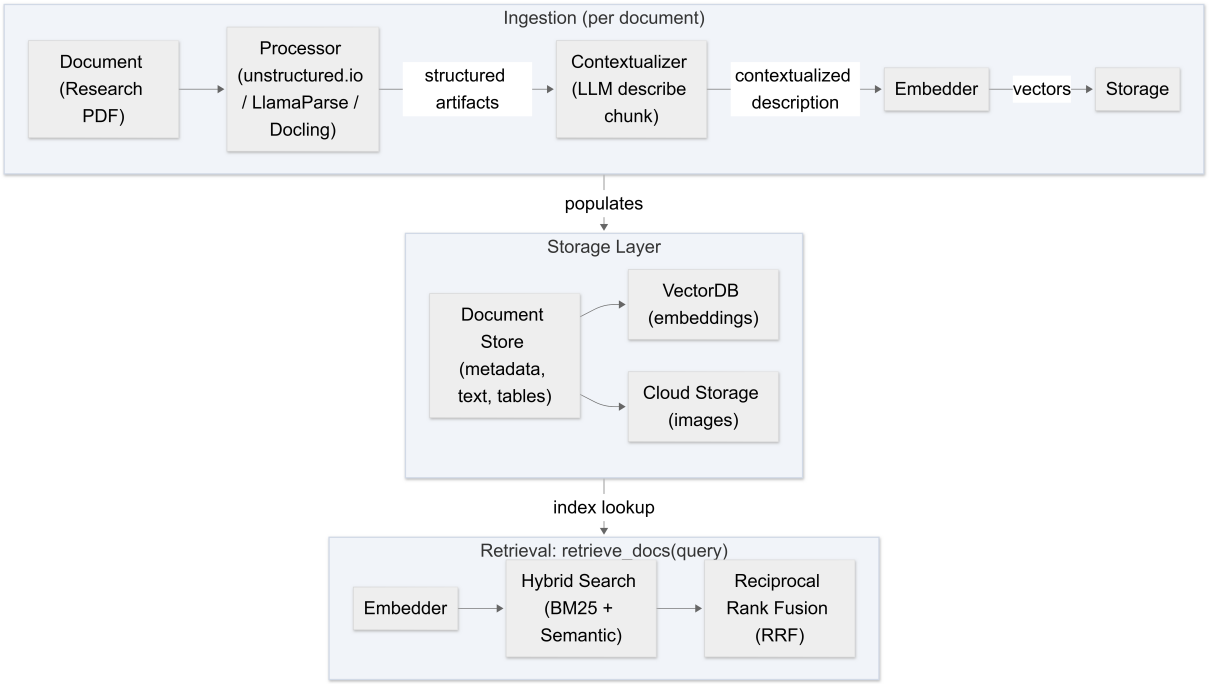

Figure 2: Full Document pipeline from ingestion, storage, to retrieval layers

deck across review cycles, promoted when severity counts improve. This protects the run from losing a better draft after a later rewrite introduces new problems. The `slide_review_events` table records critic findings, passing checks, supervisor decisions, severities, affected slides, rewrite summaries, and issue fingerprints; it is primarily used for debugging and evaluating per-cycle effectiveness before we moved evaluation to DeckBench.

Issue fingerprints give the Supervisor a simple way to track repeated failures. If the same grounding problem appears across cycles, the Supervisor can treat it as persistent instead of incidental. This is more reliable than asking a critic to remember its earlier comments in natural language, and it is cheaper than injecting full histories into every prompt. The database also leaves an audit trail of the presentation's revision process.

The stored state also helps with debugging. A failed deck is not just a final bad artifact; it is a sequence of plans, drafts, critic events, and routing decisions. For a course project, that may seem like extra machinery, but it makes the behavior inspectable. When a generated presentation is wrong, we can check whether the source was parsed badly, the plan was weak, the slide writer hallucinated, the critic missed an issue, or the Supervisor accepted too early.

3.5 Prompt and Context Scoping

The system's prompt design follows the same philosophy as its graph design: agents should receive enough context to do their job without making their role ambiguous. This is a practical response to two common LLM failures. First, when a model receives the entire paper, the whole plan, all previous comments, and every slide at once, it may answer confidently while ignoring the part of the context that actually matters. Second, when all agents see the same context, role boundaries become blurry; a slide writer may start redesigning the plan, or a critic may propose broad structural changes when only a local grounding check was requested.

The Planner receives the widest view of the source material because it is responsible for the deck's argument. Even there, the prompt is structured. The code converts raw chunks into a section outline with stable labels, so the model reasons over sections such as `S0` or `S1` rather than over database identifiers. The Planner output must use valid labels and non-empty slide blueprints. If labels are invalid or required fields are missing, the structured-output validation layer rejects the result and retries the call. This is a small but important guardrail: the plan must remain executable by the rest of the graph.

Slide Writers receive narrower context. A writer gets the blueprints for its assigned group and

the source chunks that those blueprints reference. This makes parallel drafting possible because each writer can work on a coherent group without needing the entire deck in its prompt. During rewrites, the writer also receives the current proto-slide and the specific rewrite instruction derived from critic findings. The rewrite prompt is intentionally concrete. Instead of telling the model to "make the deck better," it tells the model which slide or group has a problem and what kind of correction is required.

Critics are similarly scoped. A grounding Critic sees the slides under review and the relevant source evidence. Its job is to check whether slide content is supported. A narrative Critic sees the full deck and the plan, because coherence is a deck-level property. Separating these checks prevents a common problem in automated review, where a single critic prompt gives vague feedback like "improve clarity" without identifying whether the issue is factual, structural, or stylistic. The typed issue schema forces feedback to become actionable.

This context scoping is also important for cost and reliability. Research papers can be long, and slide generation can involve several cycles of review and rewrite. Sending every artifact to every model call would increase token usage and make rate limits more painful. By retrieving and injecting only the context required for each role, the system keeps the graph more efficient while also making each output easier to validate.

3.6 Failure Handling and Operational Controls

The project includes several practical controls because LLM workflows often fail in ordinary, uninteresting ways. A model may return malformed JSON. A slide group may come back empty. A provider may be rate-limited. A review loop may keep finding the same minor issue without producing a better deck. The system cannot eliminate these failures, but it can make them explicit and recoverable.

Structured outputs are validated with Pydantic-based schemas. When a model response does not match the expected schema, the error is written to a validation dump (for later analysis) and the call retried with the validation error included in the prompt. This makes the parts of the pipeline that are dependent on LLM calls to be resilient against individual model quirks that might not interact cleanly with pydantic schemas or json objects in general.

The graph also includes retry and budget limits. Empty first-draft slide groups are retried before review begins, but not forever. Review cycles have a configurable cap, and full replans are also limited. If the system reaches the cycle cap with unresolved issues, the Supervisor can force a replan when budget remains, accept the best-seen deck when the replan budget is exhausted, or follow debugging flags such as forced first-plan acceptance. These controls reflect a practical truth about automated generation: a system that waits for perfection may never return anything useful.

Provider routing is handled through LiteLLM instead of direct calls to one model endpoint. The application reads a YAML configuration that maps roles such as `planner`, `slides`, `critic`, `supervisor`, and `context` to model groups with fallbacks. This makes the system easier to adapt across API providers and local constraints. For a student project, this is useful because free-tier or low-cost models may fail unpredictably. The router cannot remove the need for API keys, but it keeps provider choice from being hard-coded into the agent logic.

Observability is another operational choice. The project supports Langfuse logging for graph traces and LLM calls, including transitions, latency, tokens, and model usage. These traces are not necessary for producing a deck, but they are valuable for understanding why a run behaved a certain way. When a deck is weak, the developer can inspect whether the Planner under-specified a slide, whether a Slide Writer ignored a source chunk, or whether the Supervisor accepted despite repeated major issues.

Finally, the database cache reduces repeated work. PDF parsing and the contextualization process requires a LlamaParse (paid service) call and many individual contextualization LLM calls. By hashing source PDFs and storing processed documents (and their contextualization results) the system can re-run planning and generation experiments without reparsing unchanged inputs, which facilitates iteration. A user can change the query from explanation to criticism, or adjust review-cycle settings, without paying the full ingestion cost each time.

3.7 Export Layer

The final export stage converts accepted proto-slides into a PowerPoint deck. The active builder

is `PandocBuilder`. It renders each proto-slide into Pandoc Markdown, then uses `pypandoc` with the bundled Pandoc binary to produce a `.pptx` file. This choice keeps the export path relatively simple while supporting features that research slides often need: Markdown bullets, bold emphasis, speaker notes, and LaTeX-style math.

Each slide becomes a level-one Markdown heading followed by bullet lists and optional speaker notes. Inline math and display math use Pandoc's `tex_math_dollars` extension. During PowerPoint conversion, Pandoc turns math into Office Math Markup Language, so formulas can be rendered natively in PowerPoint. This is not as visually flexible as hand-designed slides, but it is practical for technical decks where equations must survive export.

The export layer is also responsible for placing images by resolving a slide's media identifier (placed there by the Slide Writer) against the database. It checks local storage paths, object-store paths, and inline base64 data. Layout hints such as `media_left`, `media_right`, `media_center`, `two_column`, and `title_and_body` control the Markdown and Pandoc structures used for positioning. The Pandoc implementation uses a reference PowerPoint template (provided alongside the code) to assist in styling, and the user can provide their own personal reference template instead.

The main limitation is fine-grained layout control. Pandoc is reliable for converting structured Markdown into PowerPoint, but it cannot always solve text overflow or visual balance. Thus, a direction for future work: using `python-pptx` after Pandoc to apply autofit and other layout repairs. This is a reasonable tradeoff for the current project. The system focuses first on correct, grounded, editable content, while leaving high-quality visual polish as future work.

4 Evaluation

We evaluate the system using an evaluation pipeline that is kept separate from the main application runtime. The pipeline is organized into independent stages: document preparation, harness execution, grading, and report generation. This stage-separated design supports controlled evaluation by isolating sources of variance. Because each stage persists its artifacts, we can rerun grading over the same stored transcripts without regenerating decks, and we can regenerate aggregate reports from the same metric-result artifacts without recomputing model inference. The evaluation target for this project is DECKBench (Jang et al., 2026), which is a natural fit because it focuses on academic paper-to-slide generation and also treats editing and revision as important parts of the problem.

In practice, the most difficult part of evaluation has not been defining the metric pipeline, but obtaining reliable and reproducible model behavior across repeated runs. The system depends on several LLM calls during planning, drafting, critique, rewriting, and contextualization. Even when prompts and source documents are unchanged, provider-side variability, rate limits, transient API failures, and occasional malformed structured outputs can change a run enough to affect downstream benchmark scores. This is especially visible in evaluation because a small early failure can propagate into missing or weak slides later in the deck. As a result, some benchmark runs fail outright, and some successful runs are hard to reproduce exactly. This is an important learning we made: for an agentic presentation pipeline, evaluation quality depends not only on the benchmark metrics, but also on the stability of the inference stack that produces the decks.

The qualitative pattern we observed is that deck-level results tend to look better than slide-level results. This suggests that the system is relatively stronger at recovering the *global story* of a paper than at consistently producing *locally correct and polished individual slides*. In other words, the Planner and review loop usually succeed at finding a reasonable overall thesis, ordering major sections, and preserving broad coherence across the deck. However, individual slides are more vulnerable to omission, weak evidence selection, repeated context, a noisy generation call, and a relatively constrained export layer.

If time allows, a second evaluation should test revision behavior. Because the system already supports targeted rewrites, it could be adapted to DECKBench's multi-turn editing task by treating natural-language edit instructions as rewrite constraints. That test would exercise the Supervisor and the review-state tracking design directly. Until those experiments are run, this report does not claim quantitative performance improvements over Auto-Slides, PPTAgent, or DeepPresenter.

Known limitations are also part of the evaluation picture. The system depends on LlamaParse for high-quality PDF extraction, so API access and

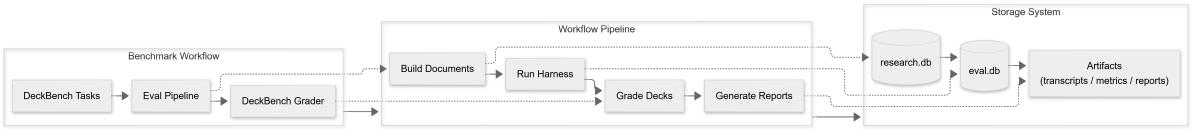

Figure 3: Full pipelines from ingestion, storage, to retrieval layers

rate limits can affect reliability. It depends on LLM providers through LiteLLM, so cost and provider failures matter. Visual design remains constrained by Pandoc and reference templates. These limits do not invalidate the architecture, but they mark the difference between a working research prototype and a polished presentation product.

5 Conclusion

This project treats research-presentation synthesis as a workflow rather than a single prompt. The Multi-Agent Research Presentation Synthesizer separates planning, drafting, critique, supervision, and export into distinct stages. A persistent database keeps source evidence, slide drafts, and review history available across cycles. That separation makes the system easier to inspect and gives each agent a narrower job.

Adding agents is only useful if the system also coordinates them. More agents can also mean more places for errors to hide. In this system, the useful pieces are role specialization, scoped context, persisted review events, and deterministic routing: a Planner focuses on narrative, Slide Writers focus on creating slide content, Critics look for errors, and the Supervisor decides whether the deck is improving. This matters when the user query changes the presentation goal from explanation to comparison or critique.

Future work should strengthen both evaluation and presentation quality. DECKBench provides a path toward systematic testing of generation and editing behavior. The export layer should also be improved with better layout repair, likely by combining Pandoc's content conversion with post-processing in PowerPoint. Finally, additional Critic review criteria and agents can be added alongside the two existing ones for additional perspectives to contribute to improving the slides (perhaps a layout-focused criteria). These improvements would move the project closer to a practical assistant for turning research into clear, audience-aware presentations.

Acknowledgments

Sheng Rao and Vien Nguyen developed the project together across the midterm and final phases, each doing roughly %50 of the work. Sheng started on implementing the multiple local document processing backends, Vien then implemented the LlamaParse backend, and Sheng finalized the integration of LlamaParse into the Planner agent. Vien started the fundamental single-responsibility multi-agent Planner-Critic-Writer loop with replan and revision reliability paths, which Sheng expanded into the full pipeline outlined in Figure 1, including expanding the loop to handle both grounding and narrative coherency critics. Sheng implemented the SQLite database used for persistent storage, research for recent related works, and the capability of Slide Writers to use figures found in the original paper(s). Vien implemented the Document Contextualizer step, the Retrieval Augmented Generation system (utilizing embeddings) used by both Slide Writers and Critics, the LangFuse logging and agent tracing system, the S3 cloud storage system, the LiteLLM dynamic model routing, and the DeckBench evaluation methodology.

References

Daesik Jang, Morgan Lindsay Heisler, Linzi Xing, Yifei Li, Edward Wang, Ying Xiong, Yong Zhang, and Zhenan Fan. 2026. DECKBench: Benchmarking multi-agent frameworks for academic slide generation and editing. *Preprint*, arXiv:2602.13318.

Yuheng Yang, Wenjia Jiang, Yang Wang, Yi Song, Yiwei Wang, and Chi Zhang. 2025. Auto-slides: An interactive multi-agent system for creating and customizing research presentations. *Preprint*, arXiv:2509.11062.

Hao Zheng, Xinyan Guan, Hao Kong, Wenkai Zhang, Jia Zheng, Weixiang Zhou, Hongyu Lin, Yaojie Lu, Xianpei Han, and Le Sun. 2025. PPTAgent: Generating and evaluating presentations beyond text-to-slides. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 14402–14418, Suzhou, China. Association for Computational Linguistics.

Hao Zheng, Guozhao Mo, Xinru Yan, Qianhao Yuan, Wenkai Zhang, Xuanang Chen, Yaojie Lu, Hongyu

Lin, Xianpei Han, and Le Sun. 2026. Deeppresenter: Environment-grounded reflection for agentic presentation generation. *Preprint*, arXiv:2602.22839.